

Exploring Data Science with CRISP-DM: A Case Study on Online Retail Data

Yugm Maneshkumar Patel
San Jose State University
`yugmmaneshkumar.patel@sjsu.edu`

October 20, 2024

Abstract

This research paper explores the application of the CRISP-DM methodology to the online retail dataset for customer segmentation. Each step of the CRISP-DM process is explained in detail, accompanied by relevant Python code. This structured approach provides insights into data preparation, modeling, and evaluation techniques. The findings from clustering highlight distinct customer segments, which businesses can leverage for marketing and decision-making strategies.

1 Introduction

The CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology is widely adopted in data science due to its structured approach to data mining projects. This paper applies CRISP-DM to an online retail dataset, analyzing customer behaviors and segmenting them using K-Means clustering. The dataset consists of transactional data, including information about invoices, quantities, and prices, which will be used to develop a model for customer segmentation.

2 Methodology: CRISP-DM Process with Code

This section describes the CRISP-DM steps applied to the online retail dataset, along with Python code snippets for each step.

2.1 Step 1: Business Understanding

Goal: Analyze the purchase behavior of customers to identify patterns, such as top-selling products or frequent buyers.

Objective: Develop customer segmentation and product recommendation systems to increase sales and improve marketing strategies.

2.2 Step 2: Data Understanding

Goal: Explore the dataset to understand its structure and identify any data quality issues.

```
import pandas as pd

# Load the dataset
data = pd.read_csv('OnlineRetail.csv', encoding='ISO-8859-1')

# Display the first few rows of the dataset
print(data.head())

# Check for missing values and data types
print(data.info())

# Basic statistics of numerical columns
print(data.describe())

# Unique values in columns
print(data.nunique())

# Explore distribution of Quantity and UnitPrice
import matplotlib.pyplot as plt
import seaborn as sns
```

```
plt.figure(figsize=(10,6))
sns.histplot(data['Quantity'], kde=False, bins=50)
plt.title('Quantity Distribution')
plt.show()

plt.figure(figsize=(10,6))
sns.histplot(data['UnitPrice'], kde=False, bins=50)
plt.title('UnitPrice Distribution')
plt.show()
```

- Loaded the online retail dataset.
- Checked for missing values and the basic structure of the data.
- Explored the distribution of key features such as 'Quantity' and 'UnitPrice'.

2.3 Step 3: Data Preparation

Goal: Clean and prepare the data for analysis by handling missing values, filtering data, and feature engineering.

```
# Drop rows with missing CustomerID
data_clean = data.dropna(subset=['CustomerID'])

# Remove negative or zero quantity values
data_clean = data_clean[data_clean['Quantity'] > 0]

# Remove rows with negative or zero UnitPrice
data_clean = data_clean[data_clean['UnitPrice'] > 0]

# Create a new feature for total transaction value
data_clean['TotalPrice'] = data_clean['Quantity'] * data_clean['
    UnitPrice']
```

```

# Convert InvoiceDate to datetime
data_clean['InvoiceDate'] = pd.to_datetime(data_clean['InvoiceDate'])

# Filter data to consider only relevant time frame (optional)
data_clean = data_clean[(data_clean['InvoiceDate'] >= '2010-12-01') & (data_clean['InvoiceDate'] < '2011-12-01')]

print(data_clean.info())

```

- Dropped rows with missing 'CustomerID'.
- Removed negative or zero quantity and price values.
- Created a 'TotalPrice' feature and ensured proper data types.

2.4 Step 4: Modeling

Goal: Develop a customer segmentation model using K-Means clustering.

Clustering - Customer Segmentation

```

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# Create RFM (Recency, Frequency, Monetary) features for segmentation
rfm_data = data_clean.groupby('CustomerID').agg({
    'InvoiceDate': lambda x: (data_clean['InvoiceDate'].max() - x.max()).days,
    'InvoiceNo': 'count',
    'TotalPrice': 'sum'
}).rename(columns={'InvoiceDate': 'Recency', 'InvoiceNo': 'Frequency', 'TotalPrice': 'Monetary'})

```

```

# Scale the data
scaler = StandardScaler()
rfm_scaled = scaler.fit_transform(rfm_data)

# Apply KMeans clustering
kmeans = KMeans(n_clusters=5, random_state=42)
rfm_data['Cluster'] = kmeans.fit_predict(rfm_scaled)

# Visualize cluster distribution
sns.countplot(x='Cluster', data=rfm_data)
plt.title('Customer Segmentation - Cluster Distribution')
plt.show()

```

Collaborative Filtering - Product Recommendation

```

from sklearn.metrics.pairwise import cosine_similarity

# Create a pivot table for user-item matrix
pivot_table = data_clean.pivot_table(index='CustomerID', columns=
    'StockCode', values='TotalPrice', aggfunc='sum').fillna(0)

# Compute similarity matrix
similarity_matrix = cosine_similarity(pivot_table)

# Get recommendations for a sample customer
customer_idx = 0 # Index for CustomerID
similar_customers = similarity_matrix[customer_idx].argsort()

[:: -1][:5]

print(f"Top 5 similar customers to customer {customer_idx}: {
    similar_customers}")

```

2.5 Step 5: Evaluation

Goal: Evaluate the performance of the clustering model using silhouette score and visualize cluster distributions.

```
from sklearn.metrics import silhouette_score

# Silhouette score for KMeans clustering
silhouette_avg = silhouette_score(rfm_scaled, rfm_data['Cluster'
    ])
print(f'Silhouette Score for customer segmentation: {
    silhouette_avg}')
```

3 Results

The customer segmentation model produced three main clusters of customers with distinct purchasing behaviors:

- **Cluster 1: High-Value Customers** - Frequent large purchases, important for high-revenue marketing.
- **Cluster 2: Occasional Buyers** - Sporadic purchases, targeted through promotions.
- **Cluster 3: Budget-Conscious Customers** - Smaller transactions, sensitive to pricing strategies.

These insights can help businesses tailor their marketing efforts, personalize promotions, and allocate resources effectively.

4 Conclusion and Future Work

In this paper, we applied the CRISP-DM methodology to the online retail dataset to analyze customer behaviors and develop a clustering model for segmentation. The model's results provide actionable insights into customer segments that businesses can leverage for marketing strategies. Future work can focus on improving the model using advanced clustering techniques like DBSCAN or incorporating more features for refined segmentation.

5 References

1. Chapman, P., et al. (2000). CRISP-DM 1.0: Step-by-step data mining guide.
2. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python, *Journal of Machine Learning Research*, 12, pp. 2825–2830.
3. Ulrik Pedersen (2019). Online Retail Dataset from Kaggle